

Pilas y colas

Tipos Abstractos de Datos: Eliminando operaciones

AED I

Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

Agenda de la clase

- 1. TADs por restricción:** qué se gana quitando operaciones
- 2. El TAD Pila (LIFO)**
Especificación, implementaciones y aplicaciones clásicas
- 3. El TAD Cola (FIFO)**
Lista enlazada, buffer circular y la cola con dos pilas
- 4. El TAD Deque**
Los dos extremos, y el problema que lo exige
- 5. Los adaptadores de la STL**
- 6. Pila o cola: cuando el TAD decide el algoritmo**

De dónde venimos

Del tema 2 damos por sabido:

El TAD como contrato: especificación frente a implementación

El invariante de la representación

Lista enlazada (simple, con último, doble circular)

Array dinámico y redimensionamiento

Coste amortizado y crecimiento geométrico

La localidad de memoria no aparece en la notación $O(\cdot)$

En el tema 2 elegíamos una representación.

Ahora elegimos qué operaciones NO ofrecer.

TADs por restricción

Quitar operaciones para ganar garantías

AED I

Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

La idea del tema

Los TADs que vamos a ver **no son estructuras nuevas**.
Son la **Lista** del tema 2 con operaciones **eliminadas**.

Y esa eliminación deliberada es lo que los hace valiosos:

1. Todo lo que queda cuesta **$O(1)$**
2. El código cliente es más fácil de razonar:
hay muchas menos cosas que pueden pasar
3. El **compilador** pasa a defender el contrato:
lo que el TAD prohíbe deja de poder escribirse

Los TADs del tema

TAD	Restricción	Coste
Pila	Solo se toca un extremo	$O(1)$
Cola	Se entra por un extremo y se sale por el otro	$O(1)$
Deque	Se entra y se sale por los dos extremos	$O(1)$

No se dice nada sobre el elemento de en medio: no es que sea caro, es que **no existe**.

TAD restrictivo

Cuando escojas una estructura, no busques la que más cosas te deja hacer, sino la **más restrictiva que todavía resuelva tu problema.**

Lo que ganas:

- Velocidad
- El compilador impide escribir el error.

El orden de salida es función del **orden de llegada**

El TAD Pila (Stack)

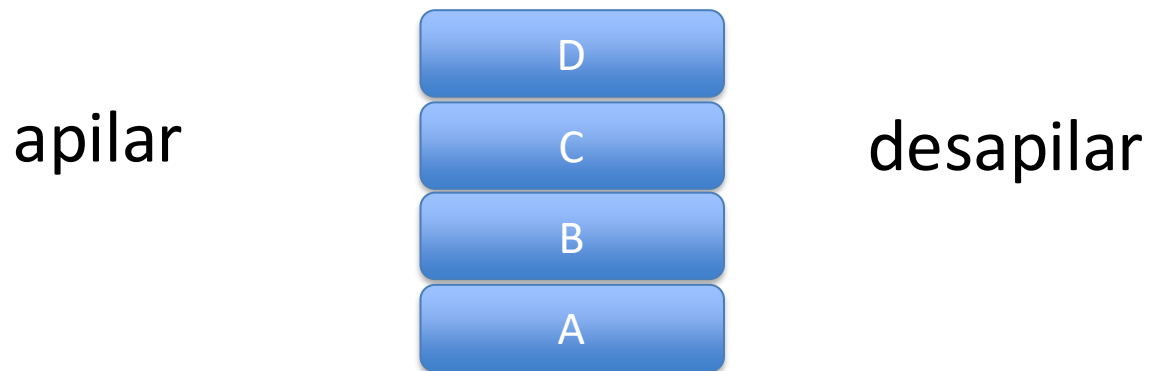
LIFO: el último en entrar es el primero en salir

AED I

Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

Pila (Stack): la especificación

Operación	Precondición	Postcondición
apilar(v) (push)	—	v pasa a ser la cima
desapilar() (pop)	—	Si no estaba vacía, elimina la cima -> true
cima() (top)	!vacía()	Devuelve la cima. No modifica
vacía() (empty)	—	Devuelve si no hay elementos
tamano() (size)	—	Devuelve el número de elementos



¿Por qué `cima()` lanza excepción y `desapilar()` no?

Es el mismo criterio del tema 2:

`cima()` devuelve un valor. Sobre una pila vacía no hay ningún valor que devolver: no hay respuesta correcta posible -> precondition, se comprueba, lanza `std::out_of_range`

`desapilar()` solo modifica. Que la pila esté vacía es un estado legítimo en tiempo de ejecución (vaciarla con un `while` es lo normal). -> sin precondition, devuelve `bool`

Y la STL hace otra cosa, deliberadamente:

`std::stack::top()` sobre una pila vacía es **comportamiento indefinido (UB)**, no una excepción. Elige no pagar la comprobación.

La cima cambia de sitio según la representación

Representación	La cima es...	Por qué
Lista enlazada	el primer nodo	Insertar/borrar por el principio es $O(1)$
Array dinámico	la última posición	Insertar al principio desplazaría n elementos

El cliente no sabe cómo se está implementando, solo lo que debe hacer el TAD. **Eso es justo lo que el TAD oculta.**

La interfaz, en C++

```
class PilaDeEnteros {           // solo QUÉ se puede hacer
public:
    virtual ~PilaDeEnteros() = default; // obligatorio
    virtual void apilar(int valor) = 0;
    virtual bool desapilar() = 0;
    virtual int cima() const = 0;
    virtual int tamaño() const = 0;

    // No virtual: se escribe una vez, para todas
    bool vacia() const { return tamaño() == 0; }
};
```

El código cliente que invierte una secuencia funciona igual con las dos implementaciones, sin enterarse de nada.

Ejercicio 1: las dos son $O(1)$... ¿y esto?

--- A través de punteros a la interfaz ---

PilaEnlazada 1000 apilar $\rightarrow$ pasos = 1000

PilaArray 1000 apilar $\rightarrow$ pasos = 2016

¿De dónde salen los 1016 pasos de más?

¿Contradice eso que apilar sea $O(1)$?

Son las copias de los redimensionamientos:

$1 + 2 + 4 + \dots + 512 = 1023$.

No lo contradice: es $O(1)$ amortizado, lo que demostramos en la clase 2. El total sigue siendo lineal.

Y aun así la de array gana casi siempre: sus pasos son copias de memoria contigua; los de la enlazada, un new por elemento.

¿Por qué la Pila aparece en todas partes?

Una pila es la respuesta siempre que aparece esta frase:

“Guardo algo **ahora** para atenderlo **luego**, y lo último que guardé es lo primero que atenderé”

Dicho de otro modo: **siempre que algo anida.**

paréntesis dentro de paréntesis $((44-33)*(55/5)-4))*2$

llamadas dentro de llamadas

operadores dentro de operadores

etiquetas HTML dentro de etiquetas HTML `<html><head>...</head><body>...</body></html>`

El anidamiento ES la estructura LIFO.

La pila de llamadas (Call stack)

La pila de llamadas de tu programa es, como su nombre indica, una pila.

Cada llamada **apila** un registro de activación:
parámetros, variables locales, dirección de retorno

Cada return lo **desapila**

Cuando una recursión infinita provoca un **stack overflow**,
lo que ha pasado es que un TAD Pila se ha quedado sin memoria.

```
3  int factorial(int n) {  
4      if (n == 0) return 1;  
5      return n * factorial(n-1);  
6  }  
7  int main() {  
8      std::cout << factorial(5) << std::endl;  
9      return 0;  
10 }
```

Corolario:

Toda recursión se puede convertir en un bucle con una pila explícita, porque es lo que el procesador estaba haciendo por ti.

CALL STACK	Paused on breakpoint
factorial!factorial(int)	factorial.cpp 4:1
factorial!factorial(int)	factorial.cpp 5:1
factorial!factorial(int)	factorial.cpp 5:1
factorial!factorial(int)	factorial.cpp 5:1
factorial!factorial(int)	factorial.cpp 5:1
factorial!factorial(int)	factorial.cpp 5:1
factorial!main	factorial.cpp 8:1

Aplicación 1: equilibrado de delimitadores

Es uno de los problemas clásicos que se resuelven de forma eficiente utilizando una pila

La regla principal: el último delimitador en abrirse debe ser el primero en cerrarse

Algoritmo:

Inicializar una pila vacía.

Recorrer la cadena de texto carácter por carácter:

Si el carácter es un **delimitador de apertura**: (, [, {, se apila.

Si el carácter es un **delimitador de cierre**:),], }:

Si la pila está vacía, la cadena **no está equilibrada** (hay un cierre de más).

Si no está vacía, se desapila el elemento superior de la pila y se comprueba si corresponde al mismo tipo de delimitador. Si no coincide, **no está equilibrada**.

Comprobación final: Al terminar el recorrido, si la pila está vacía, la cadena está **correctamente equilibrada**. Si queda algún elemento, significa que faltó cerrar algún delimitador.

Ejemplos

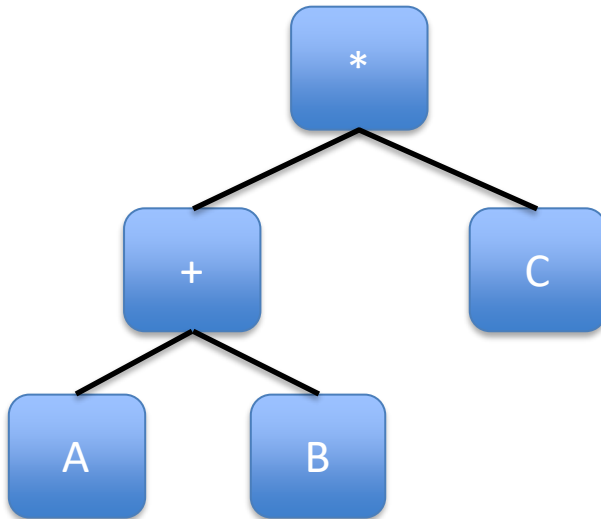
`a[i] = (b + c) * {d - e};` -> correcta

`if (v[i] > 0) { f(g(h)); }` -> correcta

`([)])` -> Mal

Aplicación 2: Evaluar notación postfija

- Para una expresión aritmética hay varias representaciones según el recorrido del árbol que la representa:



- Prefija o polaca (Preorden): $* + A B C$
- Infija (Inorden): $(A + B) * C$ (con **paréntesis**: $(A + B) * C$)
- Postfija o polaca inversa(Posorden): $A B + C *$

Aplicación 2: Evaluar notación postfija

- Ej: **Postfija**: 5 3 4 * + equivaldría a infija $5 + 3 * 4 = 17$
- Procedimiento: Recorreremos la cadena:
 - Si encontramos número lo apilamos
 - Si encontramos operador, sacamos los 2 últimos elementos y operamos y apilamos
- 5 1 2 + 4 * + 3 - $\rightarrow 5 + (1 + 2) * 4 - 3 = 14$

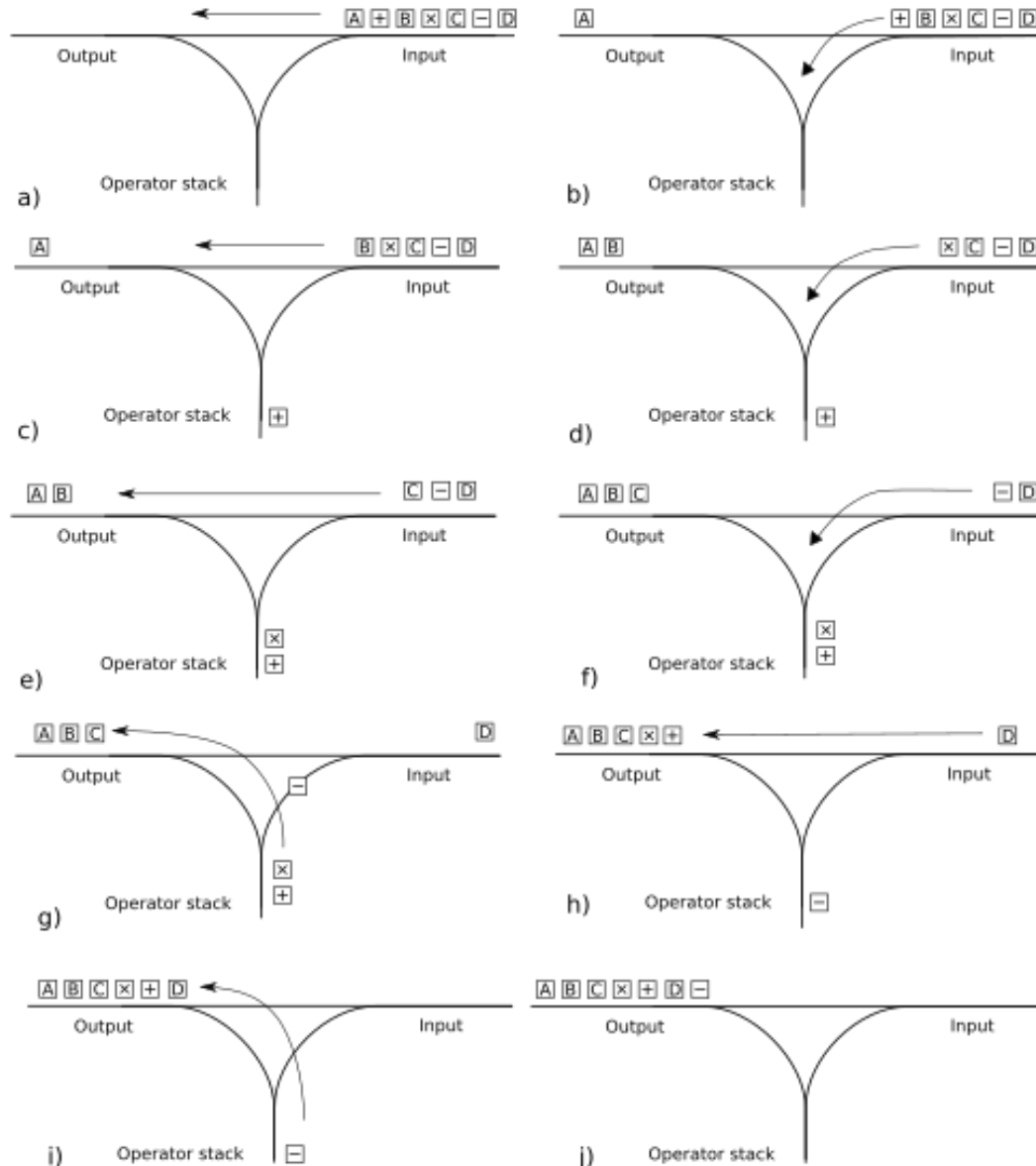


Aplicación 3: infija -> postfija

[Dijkstra](#), en 1961 trabajaba en el compilador de ALGOL 60. Necesitaba que de una pasada ($O(n)$) se procesara una operación matemática, pero las máquinas comprenden mejor la notación postfija o [notación polaca inversa](#) coloca los operadores después de sus operandos, lo que elimina la necesidad de usar paréntesis y reglas de precedencia. Este algoritmo transforma de notación infija a postfija y fue un gran avance, lo llamó **shunting-yard** (patio de maniobras en trenes)



Aplicación 3: infija -> postfija



$A+B*C-D \rightarrow ABC*+D-$

Ilustración usando un cruce de ferrocarril de tres vías.

- La entrada es procesada un símbolo a la vez, si es encontrada una variable o un número, es directamente copiado a la salida b), d), f), h).
- Si el símbolo es un operador, es colocado en el stack (push) c), e),
- Si su precedencia es menor que la del operador en el tope del stack o las precedencias son iguales y el operador es asociativo por la izquierda, entonces aquel operador es sacado del stack (pop) y añadido a la salida g).
- Los operadores restantes son sacados del stack (pop) y agregados a la salida.

https://es.wikipedia.org/wiki/Algoritmo_shunting_yard

El TAD Cola (Queue)

FIFO: el primero en entrar es el primero en salir

AED I

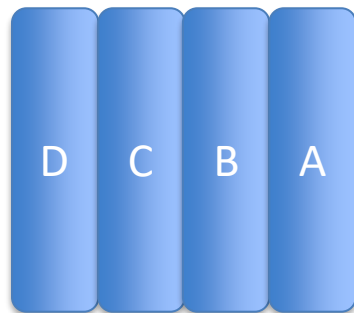
Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

Cola (Queue): la especificación

Operación	Precondición	Postcondición
encolar(v) (push)	—	v pasa a ser el último
desencolar() (pop)	—	Si no estaba vacía, elimina el primero -> true
frente() (front)	!vacía()	Devuelve el primero. No modifica
tamano() (size)	—	Devuelve el número de elementos

Mismo criterio que en la Pila: frente() lleva precondición y lanza; desencolar() devuelve bool.

encolar



desencolar

Implementación con Lista enlazada simple

Si implementamos la cola sobre una lista simplemente enlazada y **_ultimo**, hay dos formas aparentes de repartir los extremos, y solo una funciona:

Reparto de extremos	encolar	desencolar
Entrar por el final, salir por el principio	$O(1)$	$O(1)$
Entrar por el principio, salir por el final	$O(1)$	$O(n)$

Desde un nodo **no se puede retroceder**: para borrar el último hace falta el anterior. No es una preferencia, es una consecuencia.

Y aquí se ve por qué en el tema 2 nos molestamos en añadir `ultimo_` a la lista enlazada.

Aquí se aprovecha el trabajo del tema 2

```
class ColaEnlazada {  
    Nodo* primero_;    // por aquí se SALE  
    Nodo* ultimo_;     // por aquí se ENTRA  
    int  n_;  
};
```

Sin el puntero ultimo_, encolar sería $O(n)$ y la estructura no valdría absolutamente para nada.

Invariante:

```
n_ == 0 <=> primero_ == nullptr && ultimo_ == nullptr  
n_ > 0  => ultimo_->siguiente == nullptr
```


Los dos casos límite que hay que cuidar

1. Encolar sobre una cola vacía

2. Desencolar dejándola vacía

El segundo es el peligroso. Si se olvida, ultimo_ queda apuntando a memoria liberada, y el programa **no falla ahí**: falla en el siguiente encolar, que escribe por un puntero colgante.

Un fallo lejos de su causa es el más caro de depurar que existe.

03ColaEnlazada.cpp

La Cola sobre un array

Posibilidades.

AED I

Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

El problema, y la solución ingenua

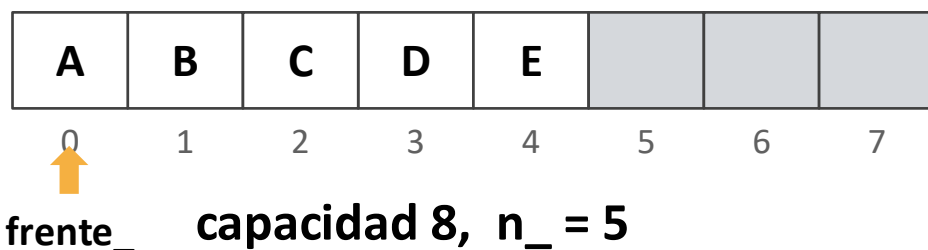
Reparto de extremos

	encolar	desencolar
Entrar por el final, salir por el principio	$O(1)$	$O(n)$
Entrar por el principio, salir por el final	$O(n)$	$O(1)$

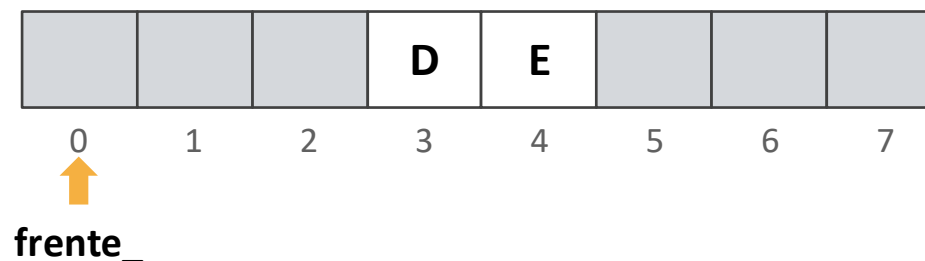
¿Y si en vez de eliminar por el principio mantenemos un índice al primero y lo movemos?

Llevaríamos un índice `frente_` que nos marca el principio de la cola. Al desencolar, solo movemos `frente_` hacia adelante:

Encolar (A); Encolar (B); Encolar (C);
Encolar (D); Encolar (E);



Desencolar (); Desencolar (); Desencolar ();
capacidad 8, n_ = 2 pero solo caben 3 más



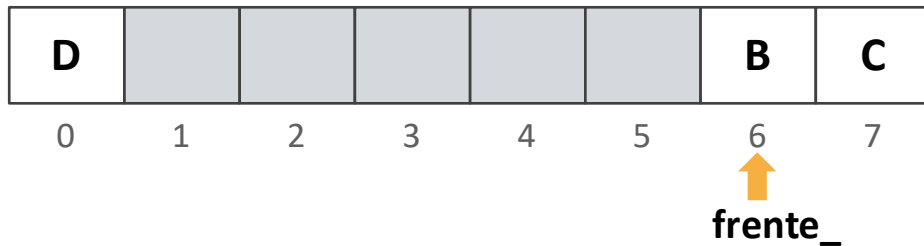
Desencolar ya es $O(1)$... pero la memoria se agota sin estar llena.

La solución buena: el array como círculo (Ring buffer)

Cuando un índice llega al final, **vuelve al 0**. ¿Cómo lo podemos hacer?

Eso es todo lo que hace el `% capacidad_`:

`n_=3; capacidad_=8; posiciones ocupadas: 6, 7, 0` — y no hay problema

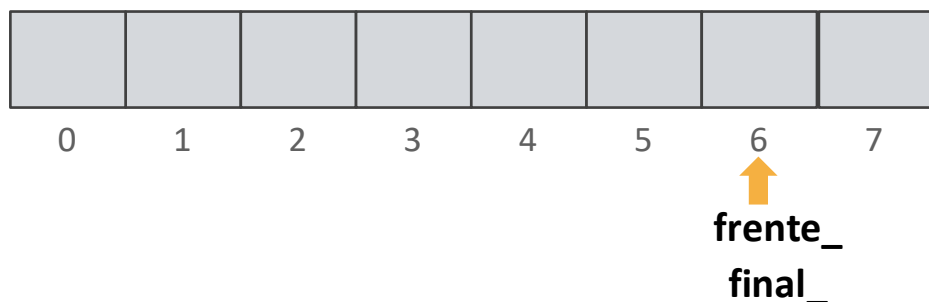


Definiremos estas dos funciones privadas:

```
int siguiente(int i) const { return (i + 1) % capacidad_; }//siguiente(6)=7;siguiente(7)=0;  
int posicionLibre() const { return (frente_ + n_) % capacidad_; }//posLibre()=(6+3)%8=1;
```

Decisión de diseño: ¿n_ o un índice final_?

Con frente_ y final_ **solamente**, vacía y llena dan la misma configuración (frente_ == final_) y no se distinguen.



Estrategia

Guardar n_ (lo que hacemos)

Dejar siempre una celda libre

Guardar un bool 'llena'

Lo que cuesta

Un entero más, y mantenerlo coherente

La capacidad útil es capacidad_ - 1

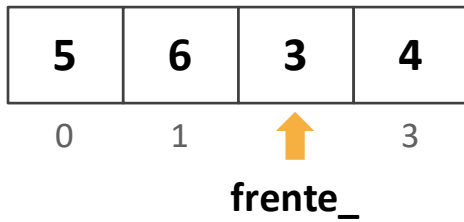
Un caso especial más en cada operación

Redimensionar: ¿cambios?

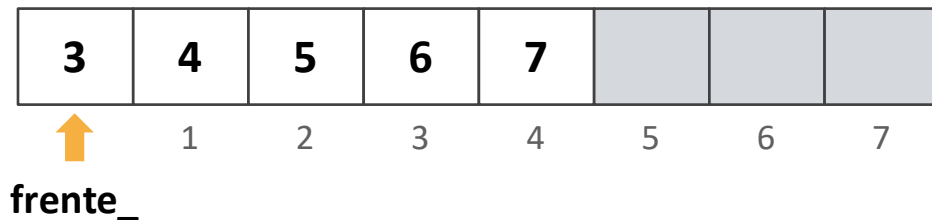
Si tenemos un array de capacidad 4 totalmente ocupado con [5 6 3 4] y `frente_ = 2`, ¿cómo encolamos 7?

Hay que recorrer **lógicamente** y escribir desde la posición 0.

llena y partida (`frente_ = 2`, cap 4)



tras encolar 7 (`frente_ = 0`, cap 8)



La variante acotada, que es la que más se usa

Si en vez de redimensionar se **rechaza** el encolar cuando está llena, tenemos un buffer de **tamaño fijo**.

Se usa en:

- driver de red: Tx y Rx de paquetes de datos entre SO y tarjeta de red. Cola llena: el driver le dice al SO que pause la transmisión temporalmente.
- buffer de audio: Tx y Rx de audio entre productor y consumidor.
 - Buffer Underflow (Vacío): El consumidor pide datos pero la cola está vacía. Provoca silencio.
 - Buffer Overflow (Lleno): El productor envía datos más rápido de lo que se reproducen. La cola se llena y se pierden paquetes.
- En general cualquier esquema productor / consumidor

Ahí el límite de memoria es un REQUISITO, no un defecto.

Se prefiere descartar un paquete a que un pico de tráfico agote la RAM del sistema.

Cola con dos Pilas: El análisis: $O(1)$ amortizado

Desencolar es **$O(n)$ en el peor caso** (cuando toca volcar) y **$O(1)$ amortizado**:

Cada elemento se mueve **exactamente cuatro veces**
en toda su vida:

1. entra en entrada_
2. sale de entrada_
3. entra en salida_
4. sale de salida_

Ni una más. Da igual cómo se intercalen las operaciones.

Una secuencia de m operaciones cuesta $O(m)$.

Ojo para sistemas de tiempo real

$O(1)$ amortizado no basta, tendríamos que ver el caso peor:

- Tras 1000 encolar: movimientos = 1000
- UNA sola llamada a frente() -> movimientos = 3000 (+2000)

Esa operación concreta es $O(n)$.

Un volcado de 1000 elementos en una sola operación puede ser **inaceptable en un sistema de tiempo real**, aunque el amortizado sea $O(1)$.

Si lo que importa es la **latencia máxima** y no el rendimiento medio, esta estructura es mala: hay que usar la cola circular.

"Peor caso" y "amortizado" responden a preguntas distintas.

El TAD Deque

Los dos extremos, y el problema que lo exige

AED I

Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

Deque: el TAD que generaliza a los otros

Double-ended queue: insertar y borrar por **los dos extremos**, todo en $O(1)$.

Usa un solo extremo -> tienes una **Pila**

Entra por uno, sale por otro -> tienes una **Cola**

Por eso `std::stack` y `std::queue` usan un **`std::deque`** por debajo.

Y además conserva algo que la lista enlazada nunca pudo dar:

Acceso por posición en $O(1)$. Aunque insertar en medio sigue siendo $O(n)$: hacer sitio es desplazar.

Implementación

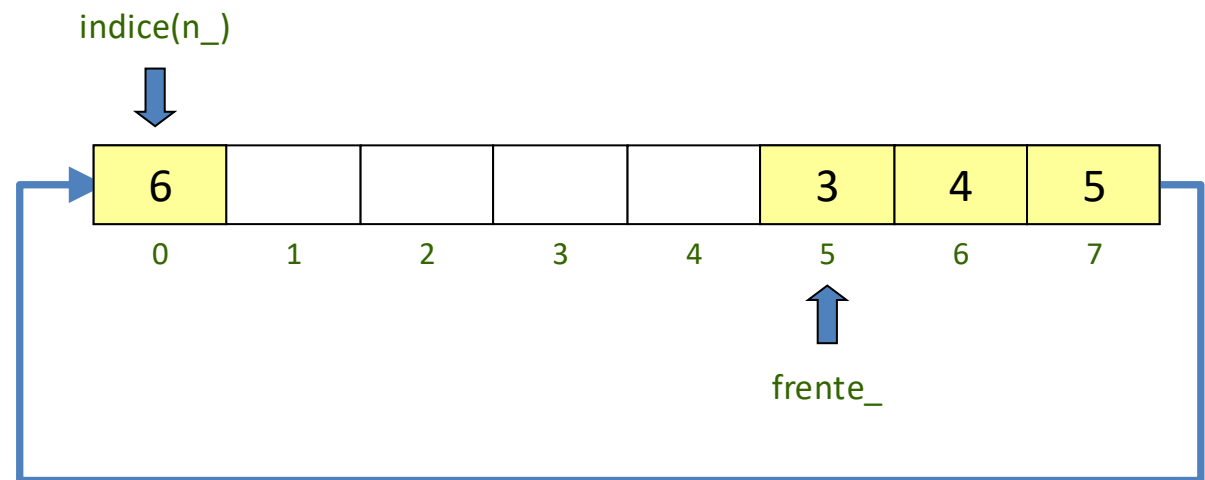
Deque sobre array circular

```
void insertarFinal(int valor) {  
    if (n_ == capacidad_)  
        redimensionar(capacidad_ * 2);  
    datos_[indice(n_)] = valor;  
    n_++;  
}
```

$O(1)$ amortizado: ni se mueve ni se recorre nada. Solo redimensionar cuesta $O(n)$, y eso se amortiza como en el tema 2.

El módulo lleva el final al principio del array sin un solo if.

insertarFinal(6)



elemento $k \rightarrow \text{datos_}[(\text{frente_} + k) \% \text{capacidad_}]$

deque: 3344556 $n_ = 34$

$\text{indice}(3) = (5 + 3) \% 8 = 0$: el final da la vuelta

```

despues 1233455 nm=345
insertarPrincipio(11, 10, 11, 12, 13, 14, 15) en el C++ fue de la vuelta! hacia atrás

```

Implementación

Deque sobre array circular

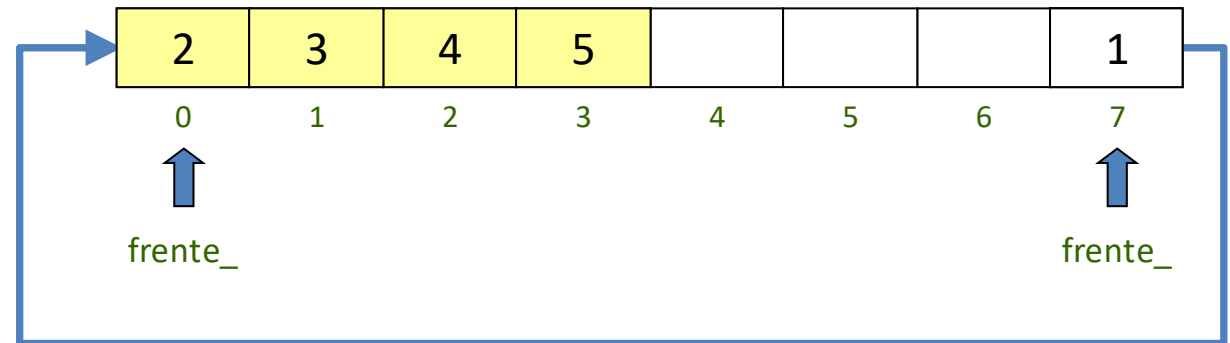
```
bool borrarPrincipio() {
    if (n_ == 0) return false;
    frente_ = siguiente(frente_);
    n_--;
    return true;
}
```

$O(1)$: avanzar un índice y restar uno.

No se borra nada del array: la casilla queda fuera del rango válido y el invariante ya no la cuenta.

Es lo mismo que desencolar en la cola circular del fichero 03.

borrarPrincipio()



elemento k $\rightarrow$ `datos_[(frente_ + k) % capacidad_]`

```
deque: 122334455   nn_=45
```

el 1 sigue $\text{siguiente}(7) = 6$ pero ya no es el de que

Implementación

Deque sobre array circular

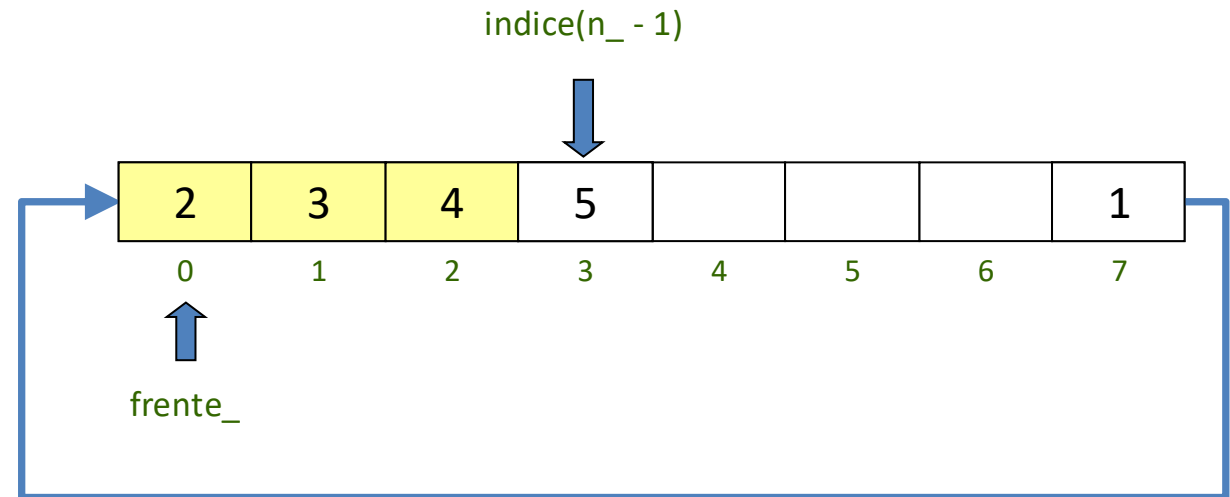
```
bool borrarUltimo() {  
    if (n_ == 0) return false;  
    n_--;  
    return true;  
}
```

La más barata de las cuatro: $O(1)$ con una sola resta.

Con los dos extremos en $O(1)$, el deque hace de pila (un solo extremo) y de cola (uno para entrar y otro para salir).

Por eso `std::stack` y `std::queue` usan un `std::deque` por debajo.

borrarUltimo()



elemento $k \rightarrow \text{datos_}[(\text{frente_} + k) \% \text{capacidad_}]$

deque: 2233445 $n_ = 3$

el último es índice (si queda se toma 1) $\% 8 = 3$

La trampa del módulo

Para insertar por delante hay que **retroceder** el índice:

```
(frente_ - 1) % capacidad_ // con frente_ == 0 da -1
```

El % de C++ NO es el módulo matemático: conserva el signo del dividendo.

La corrección es sumar la capacidad antes:

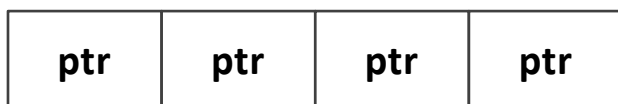
```
(frente_ - 1 + capacidad_) % capacidad_ // siempre en [0, cap)
```

Error **silencioso**: `datos_[-1]` escribe justo antes del bloque reservado, y normalmente no da error hasta que ese vecino de memoria se usa para otra cosa.

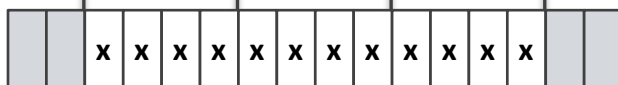
Cómo lo hace `std::deque`, que es distinto

No usa un array circular: usa un **vector de punteros a bloques** de tamaño fijo.

tabla de punteros



bloques de tamaño fijo



hueco al principio y al final

Insertar en los extremos **no invalida referencias**

`operator[]` es $O(1)$ pero con **dos direcciones**

Una deque vacía **puede reservar ya** en el constructor

Por eso `std::stack` usa deque y no vector: la STL elige lo predecible.

El problema del máximo en ventana deslizante

Máximo de cada ventana deslizante de tamaño k.

$v = [1\ 3\ -1\ -3\ 5\ 3\ 6\ 7]$, $k = 3$

máximos: 3 3 5 5 6 7

Solución ingenua: recorrer cada ventana entera $\rightarrow O(n \cdot k)$

Solución con deque: $O(n)$

El problema aparece siempre que llega **una secuencia de datos en orden** (normalmente en el tiempo) y en cada instante interesa **el extremo de lo más reciente**. Suele cumplir tres condiciones a la vez:

- los datos no dejan de llegar,
- la ventana es grande y
- la respuesta hace falta en cada paso.

Algunos ejemplos:

- **Monitorización de sistemas.** "latencia máxima de los últimos 5 minutos" o "pico de CPU en la última hora".
- **Finanzas.** Los máximos y mínimos de las últimas N sesiones.
- **Procesado de señales** o imágenes (ventanas bidimensionales).

Los adaptadores de la STL

El TAD convertido en mecanismo del lenguaje

AED I

Algoritmia y Estructuras de Datos I | Ingeniería Informática – Inteligencia Artificial

No son estructuras: son adaptadores

```
template <class T, class Contenedor = std::deque<T>>  
class stack;
```

<https://en.cppreference.com/cpp/container/stack>

La especificación es lo único que se ve, y la implementación es literalmente un argumento que puedes cambiar sin tocar tu código.

Si abres <stack> te llevas una decepción productiva: la clase entera es un contenedor miembro y una docena de funciones que delegan.

```
void push(const T& x) { c.push_back(x); }  
void pop()          { c.pop_back(); }
```

Lo más instructivo es lo que PROHÍBE

```
std::deque<int> d; d[3];    // compila
std::stack<int> p; p.top(); // compila
                p[3];     // NO COMPILA
                p.begin(); // NO COMPILA
```

Un `std::stack` **no tiene iteradores**: no se puede recorrer.

Eso **no es una carencia, es el TAD**. Sobre el deque que hay debajo sí podrías hacer `d[3]`; el adaptador lo hace imposible de escribir.

El compilador pasa a defender el contrato.

Y de ahí una regla práctica: si necesitas recorrer tu pila, lo que necesitabas **no era una pila**.

La misma pila, tres representaciones

<code>stack<int, deque<int>></code>	(defecto)	8.6 ms
<code>stack<int, vector<int>></code>		6.8 ms
<code>stack<int, list<int>></code>		96.5 ms

Las tres son $O(1)$ amortizado. Factor 14 entre la mejor y la peor. El TAD garantiza el resultado, nunca el coste.

¿Por qué el defecto es deque, si vector suele ganar?

Porque vector, al crecer, **reasigna y copia**: invalida referencias y da picos de latencia. deque crece por bloques y **nunca copia lo que ya había**.

La STL elige por defecto lo predecible, no lo más rápido.

Cada defecto responde a algo concreto

TAD	En la STL	Por defecto	Por qué
Pila	<code>std::stack</code>	<code>std::deque</code>	No copia al crecer
Cola	<code>std::queue</code>	<code>std::deque</code>	$O(1)$ por los dos extremos
Deque	<code>std::deque</code>	(es estructura)	Bloques + tabla de punteros

Tabla de costes del tema

Implementación	insertar	extraer	peor caso	memoria/int
Pila enlazada	$O(1)$	$O(1)$	$O(1)$	16 B
Pila array	$O(1)^*$	$O(1)$	$O(n)$	4-8 B
Cola enlazada	$O(1)$	$O(1)$	$O(1)$	16 B
Cola circular	$O(1)^*$	$O(1)$	$O(n)$	4-8 B
Cola con 2 pilas	$O(1)$	$O(1)$ am.	$O(n)$	8-16 B
Deque circular	$O(1)^*$	$O(1)$	$O(n)$	4-8 B

* $O(1)$ amortizado. Solo el **Deque** da además acceso por posición en $O(1)$.

La fila que casi nadie mira

El peor caso de UNA operación.

Todas las implementaciones sobre array esconden un pico $O(n)$: el redimensionamiento. El amortizado lo diluye en el total, pero **esa operación concreta tarda**.

Si escribes un **servidor web**: da igual.
Lo que importa es el rendimiento agregado.

Si escribes el control de un **airbag**: es lo único que importa. Un pico de latencia es un fallo.

Elegir entre coste medio y coste máximo no es una cuestión teórica: es la pregunta de ingeniería del tema.

Cómo elegir, en orden

1. ¿Necesito **los dos extremos**? -> Deque
2. ¿Entra por un lado y sale por el otro? -> Cola
3. ¿Solo toco **un extremo**? -> Pila
4. ¿Necesito además acceder por posición, o recorrer?
-> Entonces no era ninguno de estos: usa `std::vector`
5. ¿Tengo un **límite duro de latencia**?
-> Evita los picos $O(n)$: buffer circular de capacidad fija, con la memoria reservada por adelantado.

Resumen

Los TADs vistos son **la Lista con operaciones quitadas**.

La restricción es lo que permite que todo sea $O(1)$.

Elige **la estructura más restrictiva** que resuelva tu problema, no la que más cosas te deje hacer.

La **Pila** es la respuesta siempre que algo **anida**. Call Stack

En la **Cola** enlazada el reparto de extremos **no se elige**.

Los dos errores clásicos: encolar sobre vacía y desencolar dejándola vacía (¡anular ultimo_!).

El **buffer circular** deja todo en $O(1)$ sin desplazar nada. Al redimensionar hay que **desenrollarlo** (frente_ = 0)

La **cola con dos pilas** es $O(1)$ amortizado: cada elemento se toca **cuatro veces** exactas.

Pero su peor caso es $O(n)$: **son preguntas distintas**.

El **deque monótono** resuelve la ventana deslizante en $O(n)$.

Necesita los dos extremos: justifica el TAD él solo.